
FINAL PROJECT REPORT

Student Admission System

Individual Report

Cloud Module - Second Semester, Second Year
Technology University Dublin (TU Dublin)

AUTHOR:

X00221466 - Ivan Medel

GROUP MEMBERS:

X00221466 - Ivan Medel
X00219599 - Muhammad Abubaker
C23300041 - Daniel Ilesanmi

Table of Contents

1. Project Scenario.....	1
2. Introduction.....	1
3. Architectural Diagram and Cost Estimates.....	2
4. Iteration 1 - Single EC2 Instance with VPC.....	3
4.1 Planning.....	3
4.2 What I Implemented.....	4
5. Iteration 2 - Decoupling the App and Database.....	5
5.1 Planning.....	5
5.2 What I Implemented - and What Went Wrong.....	5
5.3 Conclusion for Iteration 2.....	7
6. Iteration 3 - High Availability and Scalability.....	7
6.1 Introduction.....	7
6.2 Planning.....	8
6.3 What I Implemented - Load Balancer.....	8
6.4 Auto Scaling and Testing (Muhammad and Daniel).....	9
7. Extra Functionalities.....	10
7.1 IAM Role (LabInstanceProfile).....	10
7.2 CloudWatch Monitoring Dashboard.....	11
7.3 RDS Automated Backups and Deletion Protection.....	11
7.4 ALB Security Group Lock Down.....	12
8. Reflection on the Ups and Downs.....	12
9. Lessons Learned.....	13
10. Future Improvements.....	13
11. Screencast.....	13
12. Final Conclusion.....	13

1. Project Scenario

This report covers the Student Admission System, a group project completed for the Cloud module at TU Dublin. The scenario given to us was a university student records website that crashes or slows down during peak admissions periods because too many people try to use it at once. The brief for the module was to move this application to AWS, which offers a scalable and highly available cloud architecture, and to build it up across three iterations.

The solution follows a traditional three-tier architecture spread across two Availability Zones within a single AWS Region: a public layer where an Application Load Balancer receives all user traffic and distributes it among web servers, an application layer of EC2 instances running Node.js on Ubuntu inside private subnets with an Auto Scaling Group managing traffic spikes automatically, and a data layer with a fully private RDS MySQL database that is never exposed to the internet.

This architecture meets the brief's requirements by scaling the Auto Scaling Group between 2 and 6 EC2 instances based on CPU load to handle thousands of users, staying highly available by spreading everything across two Availability Zones so that the application keeps running even if one zone goes down, load balancing traffic while automatically detecting unhealthy instances, keeping the database and credentials secure and off the public internet, and staying cost optimised by only paying for what is used, with idle instances automatically scaled down.

2. Introduction

This is my individual report on the Student Admission System project, completed as part of a three-person team for the Cloud module in second year, second semester at Technology University Dublin (TU Dublin). The team consisted of myself, Ivan Medel, alongside Muhammad Abubaker and Daniel Ilesanmi. While the project itself was a group effort split across three people and three iterations, this report focuses on documenting the work from my own perspective: the parts I was directly responsible for, the problems I personally ran into, and what I took away from the experience, while still giving credit to my teammates for the parts of the system they built.

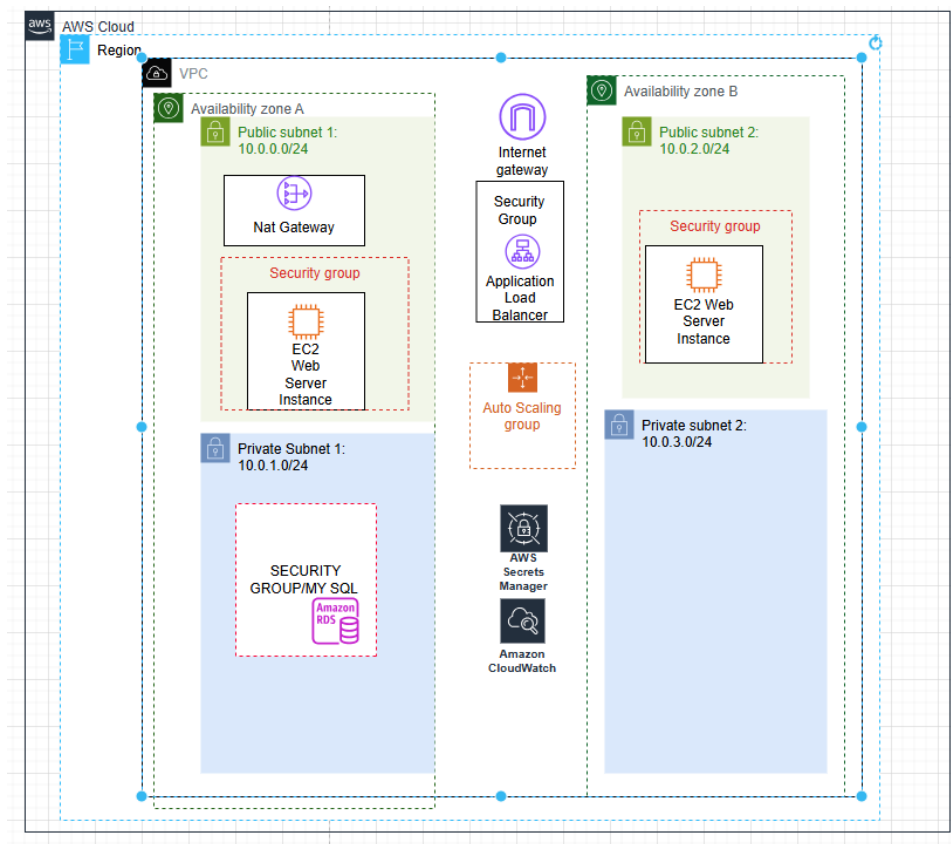
The purpose of the underlying project was to design and build a highly available, scalable web application on AWS that would improve the performance and reliability of a university's student records system, particularly during peak admission periods when thousands of users might be accessing it at the same time. To do this, we applied cloud architecture best practices based on the AWS Well-Architected Framework, including load balancing, auto scaling, secure networking, and a relational database hosted in the cloud.

The project was completed across three iterations, each one building on the last, a bit like a Jenga tower where every new piece depends on the one underneath it being solid. Iteration 1 got the application running on a single server to prove the concept worked. Iteration 2 split the database away from the application server and introduced managed AWS services to make the setup more professional. Iteration 3 then scaled the application behind a load balancer with auto scaling to handle real, unpredictable traffic. A different set of AWS services and a different set of problems showed up in each iteration, and this report follows that progression through my own experience of it.

3. Architectural Diagram and Cost Estimates

The architectural diagram was a shared effort between the three of us. Daniel was responsible for the initial layout of the diagram, Muhammad then added the security groups and refined it further, and I focused on the AWS services layer, namely Amazon RDS, CloudWatch, and the other supporting services shown in the diagram. Together we also worked out the cost estimate, coming to roughly €1,735.68 for a full 12 months of running this system.

The architecture is designed to handle peak admission traffic of around 400 to 500+ concurrent users through an auto scaling setup deployed across two Availability Zones in the EU (Ireland) region. User requests flow in from the internet through the load balancer, the web servers connect through to the RDS MySQL database, and the security layer is built from security groups that tightly control what traffic is allowed where. AWS Secrets Manager holds the database credentials, and CloudWatch is what triggers auto scaling once CPU usage climbs too high. We used a single NAT Gateway so that outbound updates could still happen while inbound internet access stayed blocked, which kept the monthly cost down without giving up much in the way of functionality.



Architecture diagram for Iterations 1 and 2, showing the VPC, subnets, and core services (Daniel, Muhammad, and Ivan)

Export Date: 03/06/2026

Language: English

[Estimate url](#)

Estimate summary

Upfront cost 0.00 USD	Monthly cost 144.64 USD	Total 12 months cost 1,735.68 USD Includes upfront cost
--------------------------	----------------------------	---

Detailed Estimate

Name	Group	Region	Upfront cost	Monthly cost
Amazon EC2	-	US East (N. Virginia)	0.00 USD	18.56 USD
Status	-			
Description:	web server instances			
Config summary	Tenancy (Shared Instances), Operating system (Linux), Workload (Consistent, Number of instances: 4), Advance EC2 instance (t2.medium), Pricing strategy (On-Demand Utilization: 100 Hours/Month), Enable monitoring (disabled), DT Inbound: Not selected (0 TB per month), DT Outbound: Not selected (0 TB per month), DT Intra-Region: (0 TB per month)			
Name	Group	Region	Upfront cost	Monthly cost
Elastic Load Balancing	-	US East (N. Virginia)	0.00 USD	17.43 USD
Status	-			
Description:	Application load balancer			
Config summary	Number of Application Load Balancers (1)			
Name	Group	Region	Upfront cost	Monthly cost
Amazon RDS for MySQL	-	US East (N. Virginia)	0.00 USD	27.25 USD
Status	-			
Description:	student database			
Config summary	Storage amount (50 GB), Storage for each RDS instance (General Purpose SSD (gp2)), Nodes (1), Instance type (db.m1.small), Utilization (On-Demand only) (100 Hours/Month), Deployment option (Multi-AZ), Pricing strategy (OnDemand), Additional backup storage (50 GB)			
Name	Group	Region	Upfront cost	Monthly cost
Amazon Virtual Private Cloud (VPC)	-	US East (N. Virginia)	0.00 USD	66.83 USD
Status	-			
Description:	nat gateway			
Config summary	Number of Regional NAT Gateways (1), Number of Availability Zones Regional NAT Gateways is active in (2), Number of NAT Gateways (0)			
Name	Group	Region	Upfront cost	Monthly cost
Amazon CloudWatch	-	US East (N. Virginia)	0.00 USD	13.52 USD
Status	-			
Description:	monitoring and alerts			
Config summary	Number of mobile OTEL events and spans or spans per visit (70), Mobile sampling rate (1), Number of Metrics (includes detailed and custom metrics) (20), Number of Dashboards (1), Number of composite alarms (10), Standard Logs: Data Ingested (5 GB)			
Name	Group	Region	Upfront cost	Monthly cost
AWS Secrets Manager	-	US East (N. Virginia)	0.00 USD	1.65 USD
Status	-			
Description:	db credentials			
Config summary	Number of secrets (2), Average duration of each secret (30 days), Number of API calls (50000 per month)			

Acknowledgement

AWS Pricing Calculator provides only an estimate of your AWS fees and doesn't include any taxes that might apply. Your actual fees depend on a variety of factors, including your actual usage of AWS services. [Learn more](#)

AWS Pricing Calculator cost estimate: €1,735.68 for 12 months

4. Iteration 1 - Single EC2 Instance with VPC

4.1 Planning

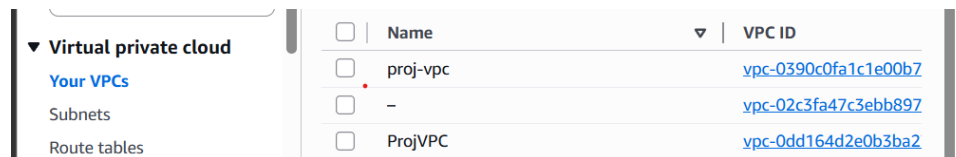
Iteration 1 had one goal: get a functional web application running on a single virtual machine and accessible from the internet. We split the work into three parts based on the layers in the diagram. Daniel took the connectivity layer, setting up the Internet Gateway and creating a security group with inbound rules for HTTP (port 80) and SSH (port 22). Muhammad handled the compute layer, launching an EC2 instance on Ubuntu with the user-data script that installs Node.js. I was responsible for the networking foundation underneath both of them.

4.2 What I Implemented

My part was getting the VPC up and running with a CIDR block of 10.0.0.0/24, creating the public subnet inside it with the CIDR 10.0.1.0/24, and enabling auto-assign public IP so our instance would get a public address once it launched. I also created the internet gateway and attached it to the VPC so traffic could flow freely, then updated the route table so it pointed to the internet gateway and associated the public subnet with it. Once that networking foundation was in place, Daniel's security group and Muhammad's EC2 instance had somewhere solid to sit, and between the three of us we had a fully functional VPC that matched the architecture diagram.

Once everything was launched, we waited for the instance to initialise, copied the public IP, and pasted it into the browser to confirm the student admission site was live on the public web.

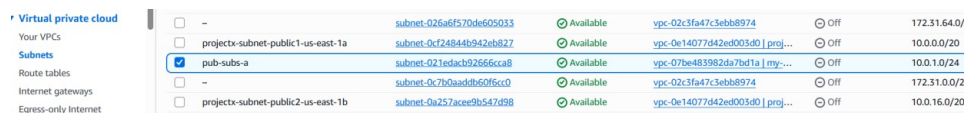
- 1 VPC "ProjVPC"
- 1 Public subnet
- 1 Internet Gateway (attached to VPC)
- 1 Route table
- 1 Security group
- 1 Running EC2 instance



The screenshot shows the AWS Virtual Private Cloud (VPC) console. On the left, there's a sidebar with 'Virtual private cloud' selected, showing 'Your VPCs', 'Subnets', and 'Route tables'. The main area displays a table of VPCs:

☐	Name	VPC ID
☐	proj-vpc	vpc-0390c0fa1c1e00b7
☐	-	vpc-02c3fa47c3ebb897
☐	ProjVPC	vpc-0dd164d2e0b3ba2

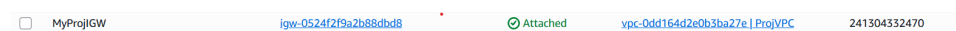
AWS Console showing the VPC (ProjVPC) created for the project



The screenshot shows the AWS Subnets console. On the left, 'Subnets' is selected under 'Virtual private cloud'. The main area displays a table of subnets:

☐	Name	Subnet ID	Status	VPC	Auto-assign public IP	Public IPv4 address pool
☐	-	subnet-026a6f570de605033	Available	vpc-02c3fa47c3ebb897	Off	172.31.64.0/20
☐	projects-subnet-public1-us-east-1a	subnet-0c724844b942eb827	Available	vpc-0e14077d42ed003d0 proj...	Off	10.0.0.0/20
☒	pub-sub-a	subnet-021edac92666cca8	Available	vpc-07be483982da7bd1a my...	On	10.0.1.0/24
☐	-	subnet-0c7b0aaddb60f5cc0	Available	vpc-02c3fa47c3ebb897	Off	172.31.0.0/2
☐	projects-subnet-public2-us-east-1b	subnet-0a257ace9b547d98	Available	vpc-0e14077d42ed003d0 proj...	Off	10.0.16.0/2

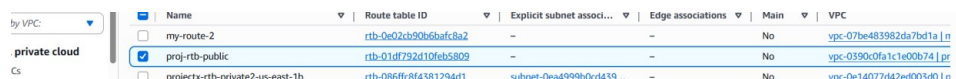
Public subnet configuration with auto-assign public IP enabled



The screenshot shows the AWS Internet Gateways console. On the left, 'MyProjGW' is selected. The main area displays a table of internet gateways:

☐	Name	IGW ID	Status	VPC	Public IPv4 address pool
☐	MyProjGW	igw-0524f2f9a2b88dbd8	Attached	vpc-0dd164d2e0b3ba2 ProjVPC	241304332470

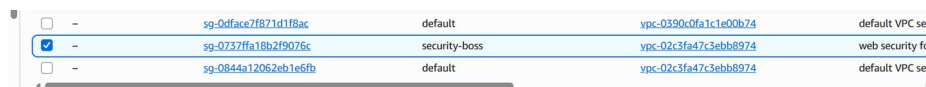
The Internet Gateway attached to the VPC



The screenshot shows the AWS Route Tables console. On the left, 'private cloud' is selected. The main area displays a table of route tables:

☐	Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
☐	my-route-2	rtb-0e02c906bafca2	-	-	No	vpc-07be483982da7bd1a n
☒	proj-rtb-public	rtb-01df732d10feb5809	-	-	No	vpc-0390c0fa1c1e00b74 pr
☐	projects-rtb-private2-us-east-1b	rtb-086ff8f4381294d1	subnet-0ea4999b0cd439...	-	No	vpc-0e14077d42ed003d0 p

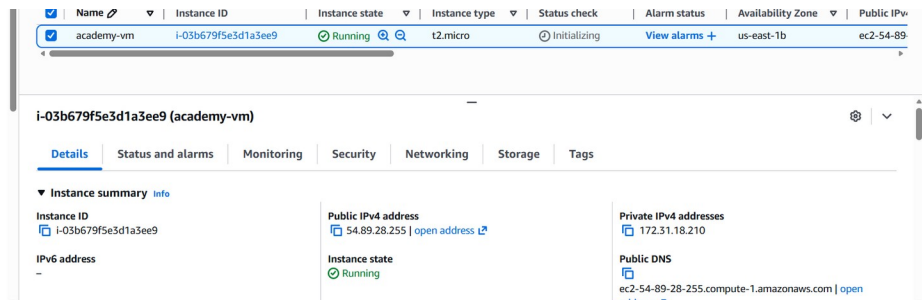
Route table showing the route to the internet gateway



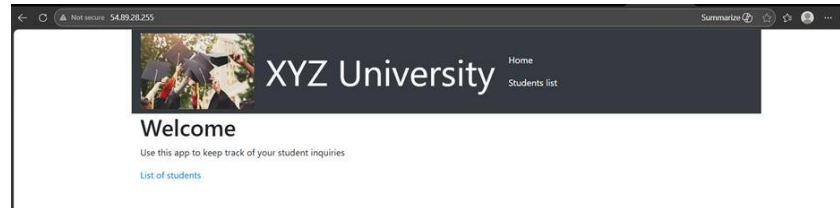
The screenshot shows the AWS Security Groups console. On the left, 'Cs' is selected. The main area displays a table of security groups:

☐	Name	Security group ID	Default	VPC	Default VPC sec.
☐	-	sg-0dface2f871d1f8ac	default	vpc-0390c0fa1c1e00b74	default VPC sec.
☒	-	sg-0737ffa18b2f9076c	security-boss	vpc-02c3fa47c3ebb8974	web security for
☐	-	sg-0844a12062eb1e6fb	default	vpc-02c3fa47c3ebb8974	default VPC sec.

Security group inbound rules for HTTP (port 80) and SSH (port 22), set up by Daniel



The running EC2 instance in the public subnet, launched by Muhammad



The student admission website running and accessible via the EC2 public IP address

5. Iteration 2 - Decoupling the App and Database

5.1 Planning

The goal of Iteration 2 was to separate the database from the web server so they could run independently. The Node.js application would stay on EC2, but the MySQL database would move to Amazon RDS as a managed service. That meant the VPC needed private subnets, and access had to be locked down so that only the EC2 instance could reach the database.

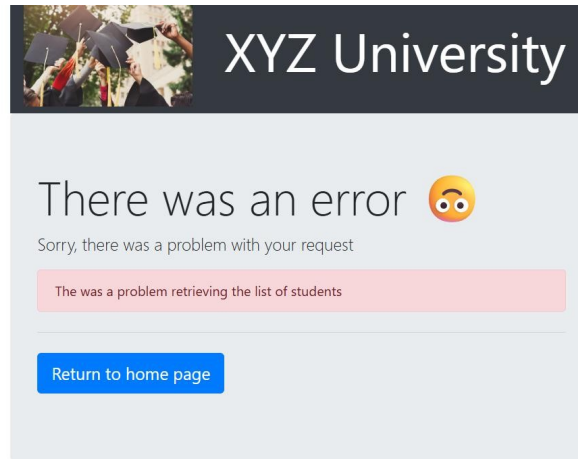
I took on the VPC updates for this iteration: adding two private subnets across two different Availability Zones, since Amazon RDS requires a DB subnet group spanning at least two AZs. These private subnets were given no route to the Internet Gateway, so the database would never be directly reachable from outside. Muhammad was on RDS setup duties, creating a security group for RDS that only accepted port 3306 from the EC2 security group and launching the MySQL RDS instance inside the private subnets I had built. Daniel then handled connecting the application to RDS, updating the environment variables, stopping the local MySQL service on the EC2 instance, and testing that everything loaded correctly with data saving to RDS.

5.2 What I Implemented - and What Went Wrong

For my part of Iteration 2, I changed the VPC by adding the two private subnets in different Availability Zones to satisfy the RDS DB subnet group requirement, then ran the database migration using mysqldump from the original EC2 instance across to the new RDS instance. What followed was about four hours of genuinely sanity-breaking errors. The application kept giving database connection errors no matter what I changed: environment variables, restarting the Node process, even just willing it to connect through sheer stubbornness didn't help.

It was only after going through every part of the AWS documentation, poring over every terminal log, and correcting misconfigured security groups and malformed IAM permissions one by one that I found the actual

problem: the Secrets Manager secret was still holding the literal placeholder text 'your-rds-endpoint' as the host value, instead of my actual RDS endpoint. Once I saw that, I took the whole thing down, restarted it with the correct values, and had it working within the hour, a stark contrast to the four hours it took to find that one wrong string among far too many subnets.

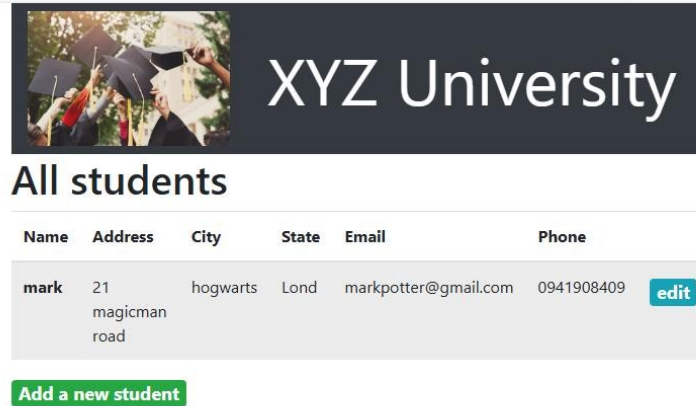


The database connection error I ran into while the Secrets Manager placeholder was still in place

Once the credentials were sorted, I could see the fix reflected immediately in the application, with records added and retrieved from the database exactly as expected.

A screenshot of a web application header for 'XYZ University' with a graduation cap icon. Below the header is a blue box with the text 'All fields are required'. The form contains several input fields: 'Name' with the value 'mark', 'Address' with the value '21 magicman road', 'City' with the value 'hogwarts', 'State' with the value 'Lond', 'Email' with the value 'markpotter@gmail.com', and 'Phone' with the value '0941908409'. Each field has a small label below it: 'Name of this student', 'Address for this student', 'City for this student', 'State for this student', 'Email for this student', and 'Phone number for this student'. At the bottom is a blue button labeled 'Submit'.

Adding a new student record to the website once RDS was correctly connected



The student admission website running successfully, connected to the RDS database

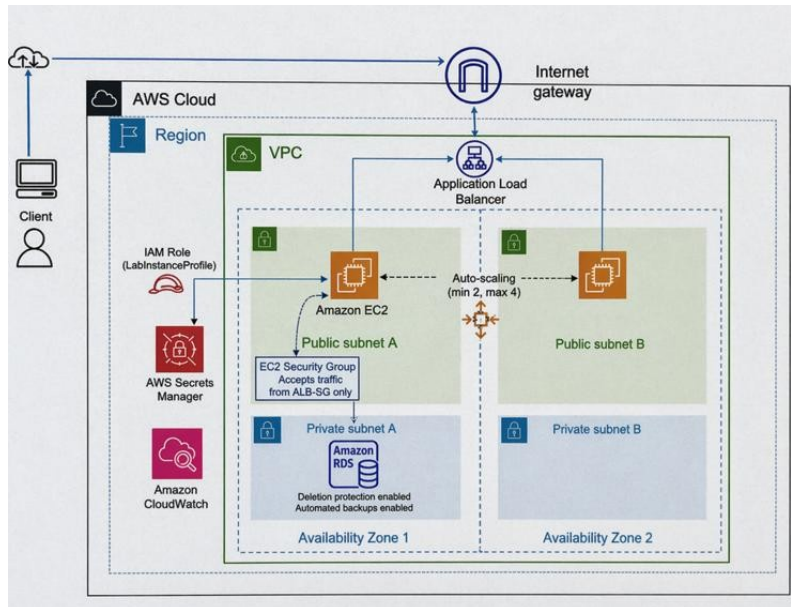
5.3 Conclusion for Iteration 2

Iteration 2 upgraded the architecture from a single-instance, break-easy model to a loosely coupled, more resilient one. The four hours I lost chasing a placeholder string in Secrets Manager was genuinely frustrating at the time, but it forced me to properly understand how EC2, IAM, security groups, and Secrets Manager all fit together rather than just clicking through a checklist. By the end of this iteration our application was sitting on a managed database, ready to scale in Iteration 3.

6. Iteration 3 - High Availability and Scalability

6.1 Introduction

In Iteration 3 we set out to make what we had already built in the first two iterations powerful enough to survive contact with the real world. Up to this point, running the application on a single server was fine for testing, but it would clearly fall over if more users showed up or the server failed. So we added a load balancer in front of the application to act as a traffic controller, and Auto Scaling to add servers automatically when traffic spiked and remove them again once things quietened down. To finish the iteration, we ran a load test to simulate a heavy burst of traffic and confirmed the application held up under the increased load.



Iteration 3 architecture diagram, showing the load balancer, auto scaling, and IAM role across two Availability Zones

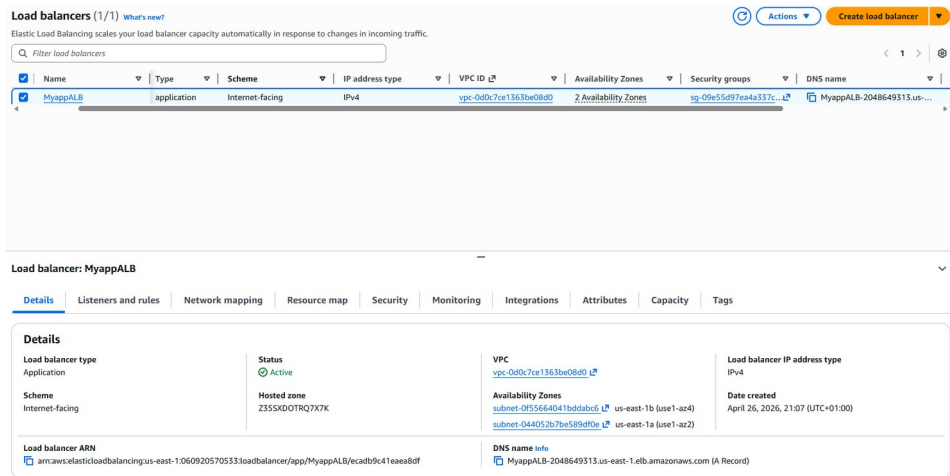
6.2 Planning

Muhammad was responsible for Auto Scaling: taking a snapshot of the working server, building an AMI from it, and configuring the Auto Scaling Group to add servers during high demand and remove them when the system was quiet, much like a restaurant kitchen calling in extra staff for a rush and sending them home afterwards, done entirely without human involvement. I was responsible for the Application Load Balancer, the 'front door' of the application, setting it up to span two Availability Zones so that if one site failed, users would not notice because the second site would keep serving requests. Daniel was responsible for testing and load testing, confirming the application ran correctly and then bombarding it with a burst of simultaneous requests to check that Auto Scaling kicked in under pressure.

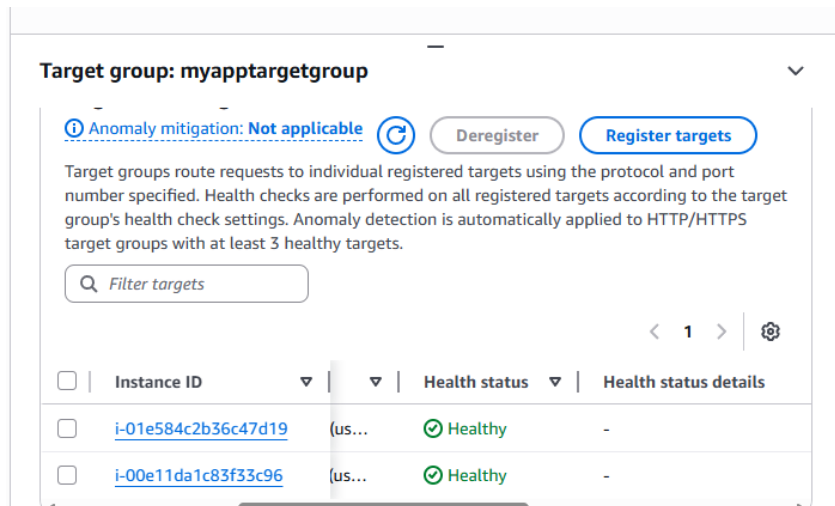
6.3 What I Implemented - Load Balancer

For this iteration I spent my time on the network and load balancer setup, which was an incremental step up from the VPC work I had done in the earlier iterations. I had to set up a second public subnet in a different Availability Zone, since the load balancer needs to span at least two zones, which took a bit of trial and error to get right. I added both public subnets to the public route table so each could reach the internet, created a new security group for the load balancer allowing inbound HTTP requests, and launched the ALB across the two subnets.

To tighten the security further, I changed the security group for the application instances (NewWebsiteAPP-SG) so it would only accept connections from the load balancer's security group rather than from the public internet directly. That meant the EC2 instances' direct IP addresses were no longer reachable at all, and the only way in was through the load balancer. I think of it as a building with just one way in, through the front door, with every side door permanently locked. Having spent four hours stuck on Secrets Manager in Iteration 2, it was satisfying to get through this iteration's networking work a good deal quicker.



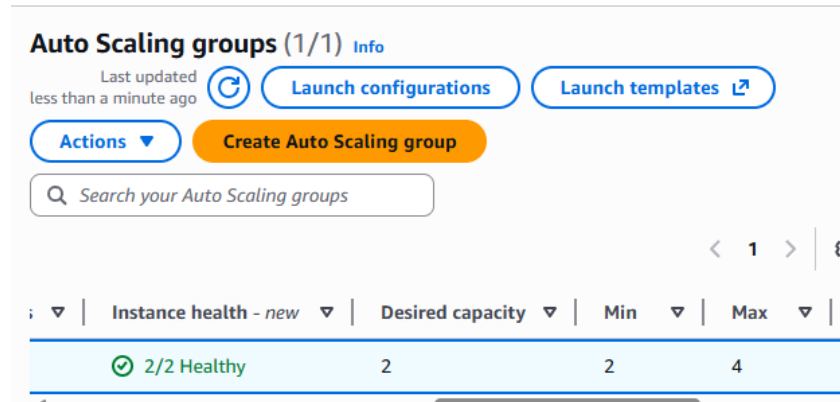
The Application Load Balancer I configured, active and spanning two Availability Zones



Healthy instances registered in the target group behind the load balancer

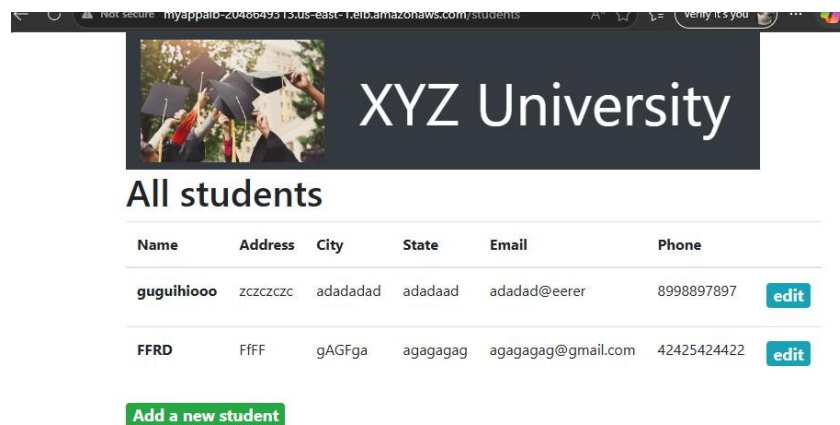
6.4 Auto Scaling and Testing (Muhammad and Daniel)

Muhammad's work getting Auto Scaling working reliably took real persistence. His main challenge was that new EC2 instances launched by the Auto Scaling Group were coming online but the application inside them was not starting, which meant health checks kept failing. He had to rebuild the AMI to make sure the application files were included correctly, and fix the IAM role so new instances could pull the database credentials from Secrets Manager during startup; without that permission, the backend couldn't log in and crashed immediately. Once resolved, he tested it properly by manually terminating a running instance to confirm the Auto Scaling Group would replace it automatically, which it did, launching a healthy replacement that connected correctly to the database and load balancer.



Auto Scaling Group configuration showing min/max capacity, set up by Muhammad

Daniel's responsibility was configuring and testing the scalability and high availability of the finished system. He connected the Application Load Balancer to a target group, configured the Auto Scaling Group using a launch template and AMI, and then ran a load test by sending repeated requests to the load balancer URL to simulate real user traffic. This caused the Auto Scaling Group to launch additional EC2 instances, confirming the system could scale horizontally, and he verified the new instances became healthy in the target group while the application stayed accessible throughout.



The application loading correctly through the load balancer's DNS name during Daniel's testing

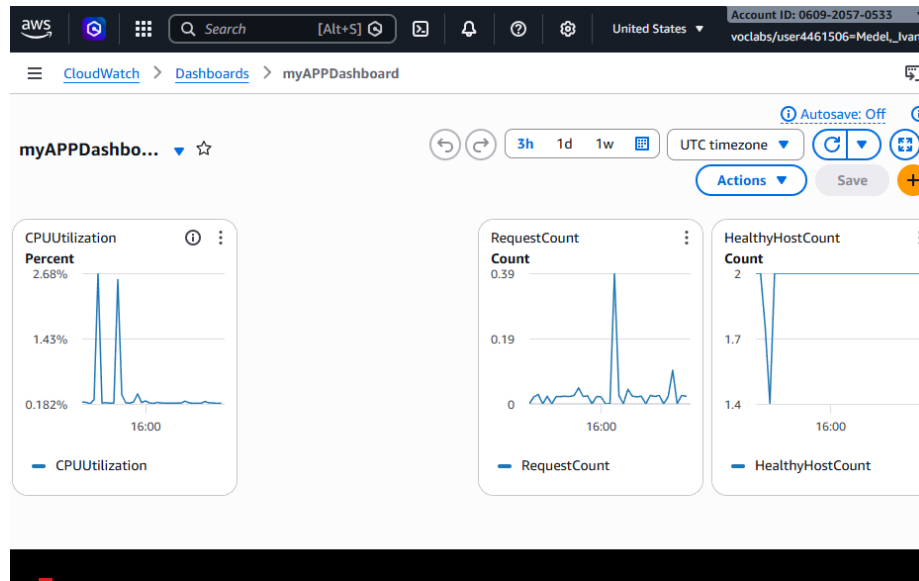
7. Extra Functionalities

7.1 IAM Role (LabInstanceProfile)

An IAM role essentially gives a server a permission slip so it is able to carry out actions in certain AWS services. Without it, the server is essentially an employee who has forgotten to bring their building pass and gets rejected from every service in the building. This specific role was needed so the server could successfully retrieve database login details during the application's startup phase. If this role were missing, the application would get blocked attempting to log in with an invalid username and password, which was the very reason it took me six hours to get everything working in Iteration 2.

7.2 CloudWatch Monitoring Dashboard

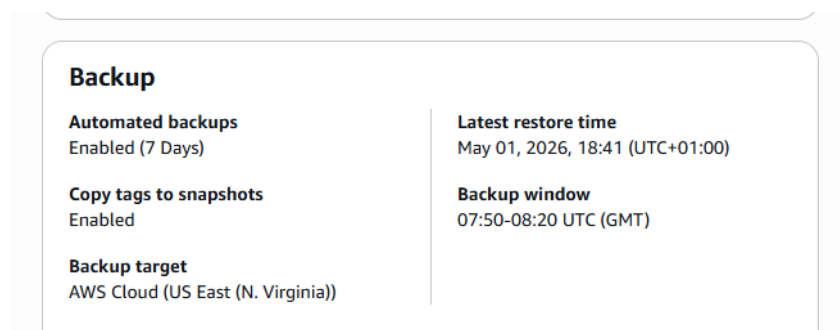
The CloudWatch monitoring dashboard displays the performance of the application in one place. Instead of clicking through several different screens, it is possible to see the number of servers currently running, how hard they are being used, and how much traffic the application is receiving. I could watch the server utilisation change dynamically as I load-tested the application and as servers were provisioned and deprovisioned in real time, which meant spending a lot less time guessing whether something was working.



The CloudWatch dashboard I built, showing CPU utilisation, request count, and healthy host count

7.3 RDS Automated Backups and Deletion Protection

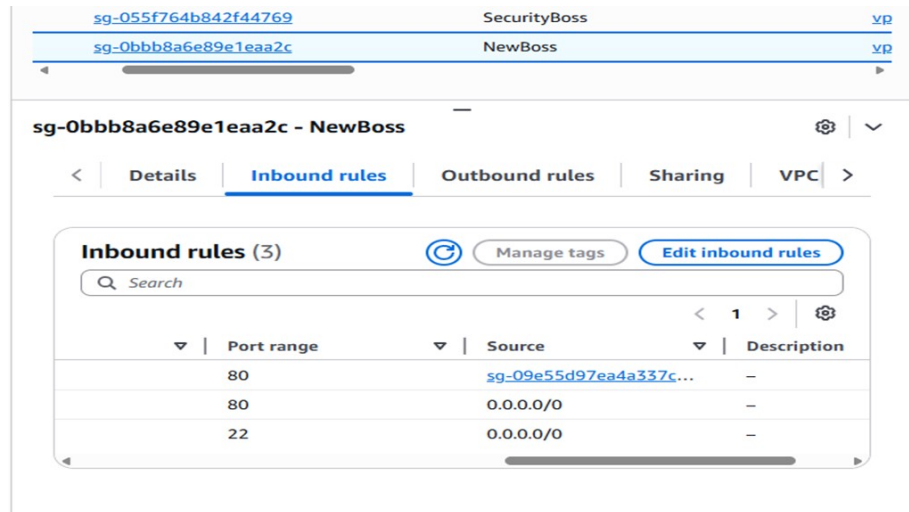
These two RDS settings are relatively minor individually, but together they contribute a large part of the system's production readiness. Automated backups take a daily backup of the database at 2 AM with a seven-day retention period, so if a worst-case scenario does arise there is always something to restore from. Deletion protection prevents the entire database from being deleted by a user with one click, an excellent safeguard against human error, in case someone accidentally deletes the entire database at 2 AM.



RDS automated backups enabled with a seven-day retention window

7.4 ALB Security Group Lock Down

With this change, direct calls to the EC2 instances are no longer possible. All traffic goes through the application load balancer, which makes the EC2 instances much more secure and no longer directly available to the internet. In other words, the back entrances to the venue are permanently locked, and you can only enter through the front door.



The application security group's inbound rules, showing port 80 only accepting traffic from the load balancer's security group

8. Reflection on the Ups and Downs

Looking back across all three iterations, the pattern that stands out most is that nearly every improvement in this project was a direct response to something failing first. The early networking uncertainty in Iteration 1 made me far more careful with route tables and subnets later on. The Secrets Manager placeholder that cost me four hours in Iteration 2 directly shaped how carefully I checked every configuration value before moving on in Iteration 3, and it made the six hours the IAM role issue took feel like a familiar kind of problem rather than a new source of panic.

The lowest point for me personally was without question those four hours in Iteration 2, staring at a database connection error that gave no honest indication of what was wrong, while I second-guessed security groups, IAM permissions, and subnet routes that all turned out to be correct. The highest point was watching the load balancer and target group in Iteration 3 show healthy instances handling real traffic, proof that the networking foundation I had been building since Iteration 1 was finally holding an entire, self-healing system up.

There were smaller ups and downs too: the quiet satisfaction of finally seeing the internet gateway attached and traffic flowing in Iteration 1, the dread of realising a single mistyped placeholder had cost an entire afternoon in Iteration 2, and the relief of getting through the Iteration 3 networking work in a fraction of the time it had taken me before, proof that the earlier frustration had taught me something.

9. Lessons Learned

- Cloud failures are often silent rather than loud: the Secrets Manager placeholder gave no useful error, so learning to check configuration values directly rather than trusting the first assumption was an important shift for me.
- IAM roles and permissions are not an abstract checkbox; a missing or misconfigured role can look identical to a dozen other kinds of failure.
- Security should be a default, not an afterthought: locking the application security group down to only accept traffic from the load balancer closed a real gap that was easy to overlook once the more visible parts of the system were working.
- Ruling things out systematically, however slow it feels in the moment, is usually faster in the end than guessing repeatedly at the same few settings.
- Working across three iterations with two teammates meant that a mistake in the networking foundation I was responsible for could block Muhammad's or Daniel's work entirely, which made me far more careful about testing my own layer before handing it off.

10. Future Improvements

Given more time, I would want to introduce Infrastructure as Code, using a tool such as AWS CloudFormation or Terraform, to remove the manual, click-through configuration that caused so much of the confusion in Iteration 2 and make the environment far easier to rebuild or roll back. I would also want to see clearer application-level error messages, so that a problem like the Secrets Manager placeholder would surface a specific, readable error instead of a silent fallback that took four hours to diagnose.

11. Screencast

For Iteration 3, I had to redo my part of the infrastructure from scratch after running out of AWS Academy budget on my original account, and recorded a fresh screencast walking through the rebuilt load balancer, target group, and networking setup on the new account.

- Ivan's Iteration 3 screencast:
<https://1drv.ms/v/c/66a729b7f55d4d44/IQBydJ488voIS6VFGOxjEx5JAUamdFOv24eds70bZKgRD-U?e=nWZaWr>

12. Final Conclusion

Across these three iterations, this project took the application from being a single, fragile server to being elastic, monitored, distributed, and genuinely built for the cloud. The database moved from sitting on the same server as the application to a fully managed RDS instance inside a private subnet. Credentials moved from being written directly into code to being held securely in Secrets Manager. The application server itself grew from a single instance into a small farm of servers behind a load balancer that scales dynamically with demand.

Along the way, I ran into real permission errors, real connectivity problems, and, in the case of that one Secrets Manager placeholder, a single wrong string that cost four hours to track down. Between the three of us, the team also ran into AWS Academy lab restrictions that blocked certain configurations outright and had to be worked around. The result is a system that is secure, resilient, and observable, and a piece of practical cloud computing knowledge that no classroom lecture on its own could have given me. Working alongside Muhammad and Daniel, each responsible for different layers of the same system, made it clear just how much a cloud architecture depends on every layer being built correctly, because when one layer breaks, as mine briefly did in Iteration 2, everyone building on top of it feels it.